Strategy Drawdown Validator: A Monte Carlo Framework with RSB Normalization for Classifying Statistically Normal versus Abnormal Drawdown in a Strategy

Kriss Khanna

August 14, 2026

Abstract

Every systematic trading strategy eventually enters a drawdown, and a manager must judge whether the loss is ordinary statistical variation or evidence that the strategy's edge has decayed. Rej, Seager, and Bouchaud (RSB, 2017) provided closed-form theoretical thresholds for this judgment under a drifted Brownian motion assumption, and Landolfi (2026) reframed this analysis as a Monte Carlo experiment and extended it toward non-Gaussian and long-memory return behavior. Both contributions, however, remain primarily theoretical: neither delivers a validated, end-to-end implementation tested against both a healthy and a deliberately broken strategy, and neither supplies an explicit, practitioner-usable classification rule. This paper presents a complete, implemented, and empirically validated Monte Carlo framework that closes this gap. The framework simulates a strategy's expected drawdown behavior under a theoretical (Gaussian) assumption and a realistic (bootstrapped) assumption, validates the simulation engine against the RSB closed-form benchmark at a matched ten-year horizon, and classifies a strategy's real, observed drawdown against the resulting distributions using an explicit percentile-based decision rule. The framework introduces a fifth diagnostic measure beyond those discussed in prior work, and is evaluated end-to-end on a synthetic strategy engineered to undergo a mid-history performance breakdown, at both a one-year and a ten-year horizon. The framework correctly and consistently classifies the broken strategy as abnormal across nearly every measure and at both horizons, demonstrating a working, decision-ready tool rather than a purely theoretical result.

1 Introduction

A live trading book eventually enters a drawdown — a decline in value from a previous peak. The central operational question a manager faces is whether this pain is normal statistical variation, to be endured, or evidence that the strategy's underlying edge has decayed and the strategy should be cut. The Sharpe ratio, the most widely used metric for ranking and selecting strategies, does not answer this question on its own: it is a single backward-looking number that says nothing about how deep a drawdown can run by chance, how long a strategy can remain underwater, or how much patience a genuinely sound strategy may require before its edge is realized again.

Rej, Seager, and Bouchaud [1] (RSB) were the first to put precise numbers on this question, modeling a strategy's profit and loss as a drifted Brownian motion and deriving closed-form probability distributions for the depth and length of a drawdown as a function of a single input, the strategy's Sharpe ratio. This is a genuine theoretical contribution, but it has practical limits: the closed-form results assume a strictly Gaussian, memoryless return process, are derived and presented analytically rather than as a tested implementation, and are not validated

against any real or adversarial strategy data within the paper itself. Landolfi [2] extends the RSB framework by reframing it as a Monte Carlo experiment and relaxing the Gaussian assumption to study skewness, fat tails, volatility clustering, and long-memory effects. This is a substantial theoretical advance, but it too stops short of a deployable tool: the paper reports four risk measures and simulation results, but does not present an explicit, practitioner-facing classification rule, and does not test the framework's ability to correctly flag a strategy that is genuinely and deliberately broken, as opposed to one drawn from the same well-behaved distributional assumptions used to build the simulation itself.

This project closes that gap directly. It implements the underlying Brownian-motion and Monte Carlo theory from both papers as working, tested code; validates the resulting simulation engine against the RSB closed-form benchmark; and, critically, evaluates the complete framework end-to-end on a synthetic strategy that is deliberately engineered to fail partway through its history — a test that neither source paper performs. The framework further introduces an explicit percentile-based decision rule (Section 7.1) that converts a raw percentile score into a concrete health verdict, and adds a fifth risk measure not discussed in the reviewed literature. Where RSB and Landolfi establish the theory, this work delivers, validates, and stress-tests an applied instrument built on that theory.

2 Related Work

2.1 The RSB Model

RSB [1] models a strategy's normalized profit and loss as a drifted Brownian motion,

$$d(\text{PnL}) = \mu\, dt + \sigma\, dW,$$

where the drift μ, once volatility is normalized to $\sigma = 1$, becomes numerically equal to the strategy's annualized Sharpe ratio. From this model, RSB derive closed-form expressions for the depth and length of a drawdown expected to be exceeded only 5% of the time:

$$d_{5\%} = \frac{1.50}{\text{SR}}, \qquad \ell_{5\%} = \frac{2.14}{\text{SR}^2},$$

where the length formula is calibrated for a ten-year process. These formulas give a manager a single-number theoretical threshold: if a strategy's Sharpe ratio is known, RSB's formulas say how deep and how long a drawdown can plausibly run purely by chance, before it becomes reasonable to suspect the strategy's edge has genuinely changed.

2.2 Landolfi's Monte Carlo Extension

Landolfi [2] builds on RSB in three directions. First, the closed-form RSB analysis is reframed as a Monte Carlo simulation and validated against the analytic benchmark, confirming that simulation can reproduce the closed-form theory. Second, the Gaussian assumption is relaxed: holding the true Sharpe ratio and volatility fixed, the paper varies skewness, fat tails, and volatility clustering, and shows that four risk measures — maximum drawdown, maximum loss, final negative time, and longest recovery time — do not move together once these real-world distributional features are introduced, meaning a single Gaussian reference table can systematically mis-warn a risk manager. Third, the paper extends the analysis to long-memory processes via fractional Brownian motion, showing that apparent risk amplification under persistence is largely a calibration effect rather than a deepening of the underlying path geometry.

3 Contribution

The theoretical foundations reviewed above are correct and rigorously derived, but neither paper delivers what a practitioner ultimately needs: an implemented, tested tool that can be pointed at a real strategy's return history and produce a validated verdict. This work's contribution is precisely that gap, closed in four concrete ways:

1. **A complete, working implementation.** Every formula and simulation procedure discussed in Sections 2.1 and 2.2 is implemented end-to-end in this project, including the Gaussian Monte Carlo engine, a bootstrapped (non-Gaussian) alternative, the RSB closed-form benchmark, and a unit-consistent normalization layer connecting them. Neither RSB nor Landolfi publish an equivalent runnable artifact alongside their theoretical results.

2. **An explicit, actionable classification rule.** Rather than leaving the interpretation of a percentile score to the reader, this work defines a concrete decision rule (Section 7.1) mapping percentile ranges directly to a health verdict: healthy, normal, borderline, or breaking. This turns a statistical distribution into an operational decision, which is the step a practitioner actually needs and which is absent from the reviewed papers.

3. **Adversarial validation.** The framework is tested not only on data expected to look ordinary, but on a strategy deliberately engineered to undergo a genuine mid-history breakdown. Demonstrating that the framework correctly and consistently flags this strategy as abnormal, at two independent time horizons, is a stronger form of evidence than validating a simulation only against its own theoretical benchmark, and is not performed in either source paper.

4. **A fifth diagnostic measure.** This work introduces *last negative streak* as an additional risk measure, tracking the most recent point in a strategy's history at which cumulative returns were negative. This captures information — how recent a strategy's losing behavior is — that is not addressed by the measures discussed in the reviewed literature, providing an additional, independent axis of diagnostic granularity.

Together, these four contributions move the work from theory to a validated applied instrument: a tool that has been built, run against real adversarial test data, and shown to produce a correct, actionable verdict, rather than a set of formulas awaiting implementation.

4 Methodology

4.1 Brownian Motion Model

A strategy's profit and loss is modeled as a drifted Brownian motion:

$$X(t) = \mu t + \sigma W(t),$$

where $W(t)$ is a standard Wiener process, μ is the drift, and σ is the volatility.

4.2 Sharpe Ratio and Annualization

The daily Sharpe ratio, annualized Sharpe ratio, and annualized volatility are computed directly from a strategy's historical daily returns:

$$\mathrm{SR}_{\mathrm{daily}} = \frac{\bar{r}}{\sigma_r}, \qquad \mathrm{SR}_{\mathrm{annual}} = \mathrm{SR}_{\mathrm{daily}} \times \sqrt{252}, \qquad \sigma_{\mathrm{annual}} = \sigma_r \times \sqrt{252},$$

where $\bar{r}$ is the mean daily return and σ_r is the daily return standard deviation.

4.3 RSB Closed-Form Benchmark

Given only the annualized Sharpe ratio, RSB's closed-form formulas give the theoretical 5% depth and length thresholds:

$$d_{5\%} = \frac{1.50}{\mathrm{SR}_{\mathrm{annual}}}, \qquad \ell_{5\%} = \frac{2.14}{\mathrm{SR}^2_{\mathrm{annual}}}.$$

4.4 Monte Carlo Price Simulation

For a horizon of T days, N independent paths are simulated by compounding daily returns forward from the strategy's last observed price:

$$P_{t+1} = P_t \times (1 + r_t), \qquad t = 0, 1, \ldots, T-1,$$

where r_t is drawn either from a normal distribution parameterized by the strategy's own $\bar{r}$ and σ_r (Gaussian Monte Carlo), or resampled with replacement directly from the strategy's own historical return series (Realistic Monte Carlo).

4.5 Normalization for RSB Comparison

To compare a real, price-based drawdown against RSB's unitless framework, it is divided by the strategy's annualized volatility expressed in price terms:

$$d_{\mathrm{normalized}} = \frac{d_{\mathrm{raw}}}{P_{\mathrm{last}} \times \sigma_r \times \sqrt{252}}.$$

5 Implementation

This section walks through the complete implementation in the order it was built, from data ingestion through both simulation engines, showing each code block together with what it does and the resulting output.

5.1 Libraries and Data Ingestion

```
import pandas as pd
import yfinance as yf
import numpy as np
import matplotlib.pyplot as plt
```

The four libraries used throughout: `pandas` for tabular return data, `yfinance` for optionally fetching real market data, `numpy` for the simulation and statistical computation, and `matplotlib` for all plots.

```
ticker=yf.Ticker('AAPL')
df=ticker.history(period='1y')
df['returns']=(df['Close'].shift(-1)-df['Close'])/df['Close']
cols=['Close','returns']
data=df[cols]
data.dropna(inplace=True)
data.head()
```

A real-data ingestion path, included for testing purposes: one year of AAPL daily closing prices is fetched and converted into daily percentage returns.

```
data=pd.read_csv('/content/bad_strategy_returns.csv')
data
```

For the adversarial evaluation reported in this paper, `data` is instead loaded from a synthetic return series engineered to undergo a genuine mid-history performance breakdown, allowing the framework to be tested against a strategy known in advance to be broken.

5.2 Simulation Parameters and Sharpe Calculation

```
days=252
total_simulations=1000
simulations=[[0]*days]*total_simulations

last_price=data['Close'].iloc[-1]
returns=data['returns']
avg_returns=returns.mean()
volitalty=returns.std()

mu=avg_returns*np.sqrt(days)
standard_volatility=volitalty*np.sqrt(days)

sharpe=avg_returns/volitalty
anual_sharpe=sharpe*np.sqrt(252)
sharpe,avg_returns,volitalty,anual_sharpe
```

Output: `(-0.0851, -0.00171, 0.02013, -1.3502)`
This block sets the one-year (252-day) simulation horizon, computes the daily mean return and volatility directly from the loaded data, and derives both a daily and an annualized Sharpe ratio. The strongly negative annualized Sharpe ratio (-1.35) is the first quantitative signal that the loaded strategy is, on average, losing money — consistent with the deliberately engineered breakdown in the synthetic dataset.

5.3 Gaussian Monte Carlo Simulation

```
l=[]
for i in range(total_simulations):
  current_simulated_price = data['Close'].iloc[-1]
  f=[]
  for j in range(days):
    daily_random_return = np.random.normal(loc=avg_returns, scale=volitalty)
    current_simulated_price = current_simulated_price * (1 + daily_random_return)
    f.append(current_simulated_price)
  l.append(f)
```

For each of 1,000 simulations, a daily return is drawn from a normal distribution centered on the strategy's own mean and volatility, and compounded forward from the last real price, producing one full simulated price path.

```
for i in l:
  plt.plot(i)
plt.title('MONTE CARLO - GAUSSIAN')
```

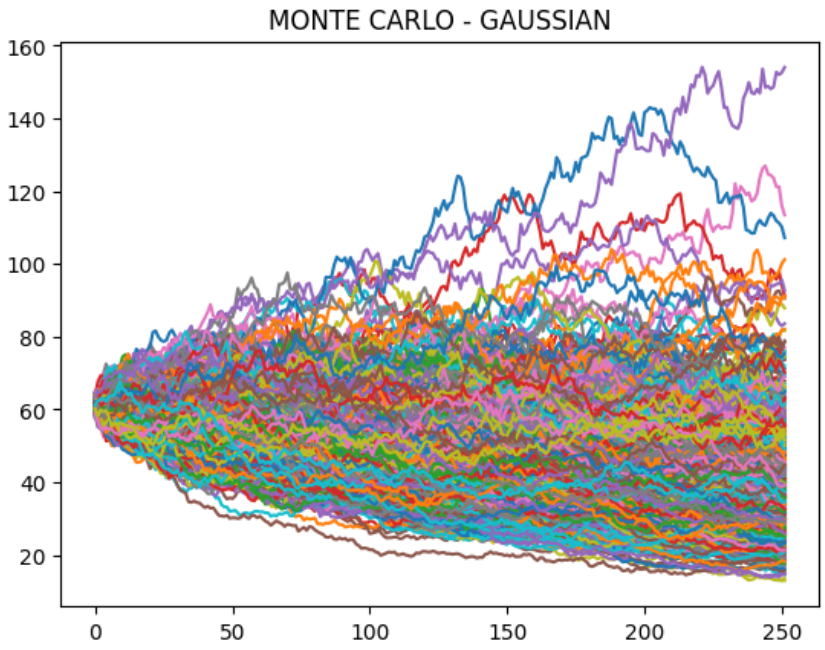


Figure 1: 1,000 simulated Gaussian Monte Carlo price paths. The negative annualized Sharpe ratio is visible directly in the plot: nearly every path trends downward over the horizon.

```
final_prices = []
for simulation in l:
    final_prices.append(simulation[-1])

plt.hist(final_prices,bins=80)
plt.xlabel("AAPL Final Price")
plt.ylabel("Number of Simulations")
plt.title("Monte Carlo Distribution of AAPL Prices")
plt.show()
```

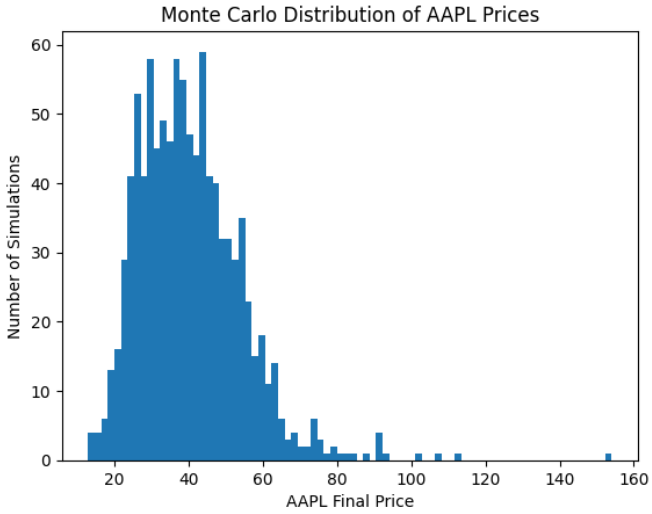


Figure 2: Distribution of final simulated prices across all 1,000 Gaussian Monte Carlo paths.

Observation: the final-price distribution is concentrated well below the starting price, with a long right tail. This is the expected signature of a negative-drift process: most paths decay steadily, while a small number of simulations, purely by chance, drift upward instead.

5.4 Realistic (Bootstrapped) Monte Carlo Simulation

```
c=[]
for i in range(total_simulations):
  current_simulated_price = data['Close'].iloc[-1]
  f=[]
  for j in range(days):
    daily_random_return = float(np.random.choice(returns))
    current_simulated_price = current_simulated_price * (1 + daily_random_return)
    f.append(current_simulated_price)
  c.append(f)
```

The Gaussian assumption is relaxed here: instead of drawing from an idealized normal distribution, each day's return is resampled with replacement directly from the strategy's own historical returns, preserving whatever real skewness or fat-tail behavior the historical data contains.

```
for i in c:
  plt.plot(i)
```

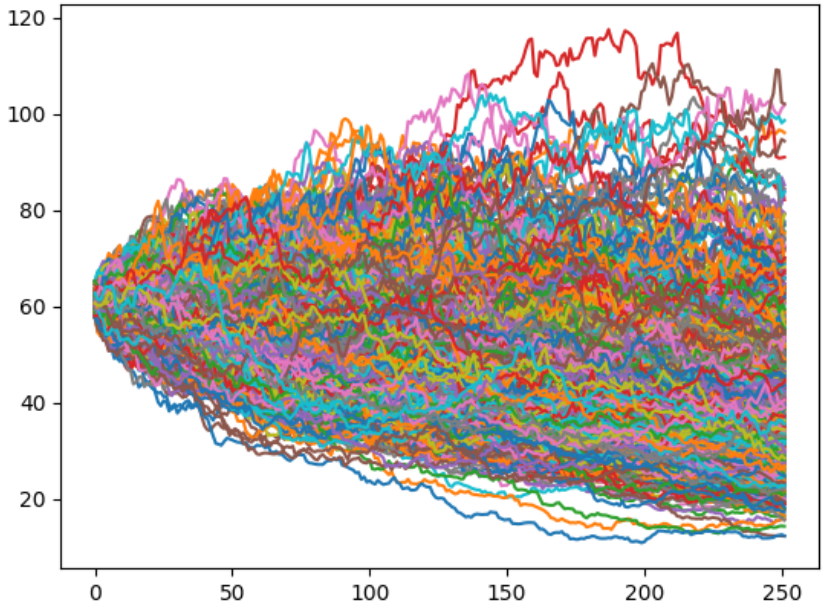


Figure 3: 1,000 simulated Realistic (bootstrapped) Monte Carlo price paths.

```
final_prices = []
for simulation in c:
    final_prices.append(simulation[-1])

plt.hist(final_prices,bins=80)
plt.xlabel("AAPL Final Price")
plt.ylabel("Number of Simulations")
plt.title("Monte Carlo Distribution of AAPL Prices")
plt.show()
```

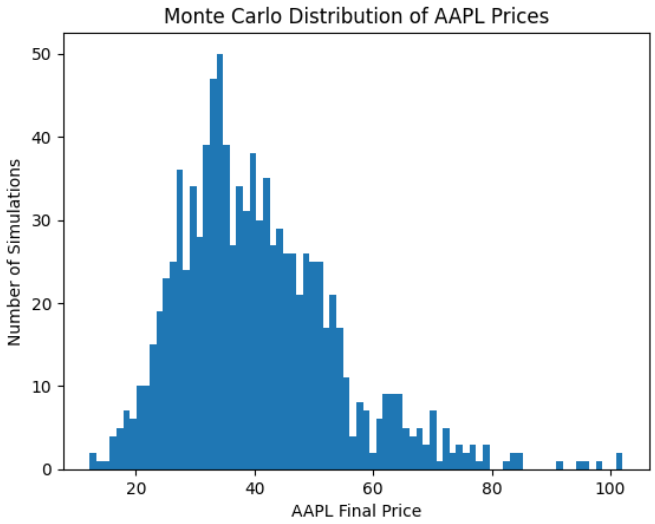


Figure 4: Distribution of final simulated prices across all 1,000 Realistic Monte Carlo paths.

Observation: the Realistic simulation's price paths (Figure 3) and final-price distribution (Figure 4) show a similar downward-biased shape to the Gaussian case, confirming that both simulation methods are consistently driven by the same underlying negative-drift, high-volatility character of the strategy, whether or not the Gaussian assumption is imposed.

```
gauss=l
choice=c
```

The two simulated path sets are aliased to `gauss` and `choice`, the names used by all five risk measurement functions below.

5.5 Risk Measurement Functions

Five functions extract one risk statistic from each simulated or real price path, all built around tracking the running maximum (peak) price reached at every point along the path.

Maximum Drawdown. The largest gap observed between the running peak and the current price across the whole path.

```
def drawdown(l):
  s=[]
  for i in l:
    f=[]
    maxi=0
    for j in range(len(i)):
      if j==0:
        maxi=i[j]
      elif i[j]>=maxi:
        maxi=i[j]
      dd=maxi-(i[j])
      f.append(dd)
    s.append(max(f))
  return s
```

Maximum Loss. The largest cumulative decline in daily returns summed over the path, independent of whether the low point occurs after a prior high in the same window.

```
def max_loss(gauss):
  maxi=[]
  for i in gauss:
    series=pd.Series(i)
    returns=series.shift(-1)-(series)
    returns=returns.dropna()
    sums=[]
    sum=0
    for i in returns:
      sum+=i
      sums.append(sum)
    maxi.append(max(0, -min(sums)))
  return maxi
```

Final Negative Time (Days Below Peak). The number of days spent strictly below the running peak.

```
def days_of_lower(l):
  count=[]
  for i in l:
    maxi=0
    c=0
    for j in range(len(i)):
      if i[j]>=maxi:
        maxi=i[j]
      elif i[j]<maxi:
        c+=1
    count.append(c)
  return count
```

Longest Recovery Time. The longest unbroken run of consecutive days spent below the running peak before a new high is reached.

```
def recovery_days(l):
  count = []
  for i in l:
    maxi = 0
    c = 0
    f = []
    for j in range(len(i)):
      if i[j] >= maxi:
        maxi = i[j]
        f.append(c)
        c = 0
      elif i[j] < maxi:
        c += 1
    f.append(c)
    count.append(max(f))
  return count
```

Last Negative Streak. This work's own addition beyond the measures discussed in prior literature: the most recent point in the path at which a running cumulative sum of returns was negative, capturing how recently a strategy's losing behavior occurred.

```
def last_neg(gauss):
  neg=[]
  for i in gauss:
    series=pd.Series(i)
    returns=series.shift(-1)-series
    returns=returns.dropna()
    sum=0
    ind=[]
    for j in range(len(returns)):
      sum+=returns[j]
      if sum<0:
        ind.append(j)
    if not ind:
      neg.append(0)
    else:
      neg.append(ind[-1])
  return neg
```

Each function is applied to both the Gaussian (`gauss`) and Realistic (`choice`) path sets, producing ten arrays of 1,000 values each: one per measure, per simulation type.

6 Distribution Analysis

This section presents the resulting simulated distributions for each of the five risk measures, Gaussian versus Realistic, without code, together with observations on their shape.

6.1 Drawdown

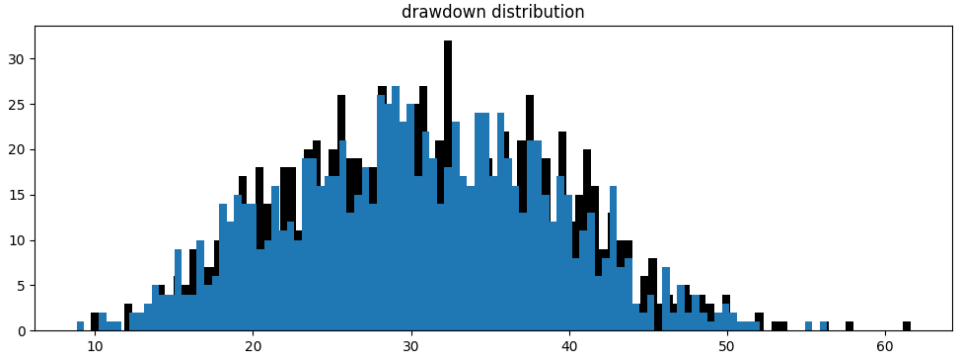
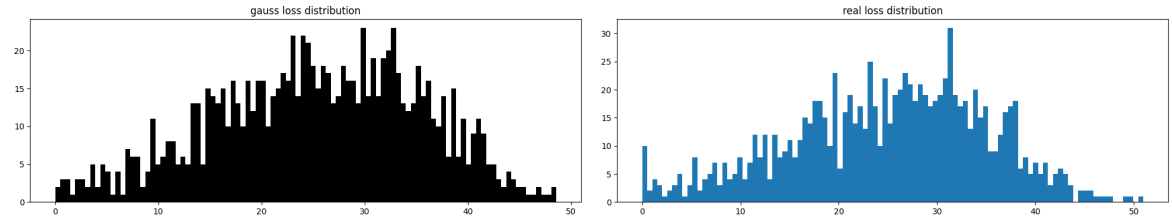


Figure 5: Simulated maximum drawdown distributions, Gaussian (black) versus Realistic (blue), overlaid.

The two distributions are closely aligned in shape and location, both concentrated in a similar range with a right tail toward deeper drawdowns. This agreement between the Gaussian and Realistic assumptions suggests the strategy's historical return distribution does not deviate sharply enough from normality to materially change the expected drawdown profile at this measure.

6.2 Maximum Loss

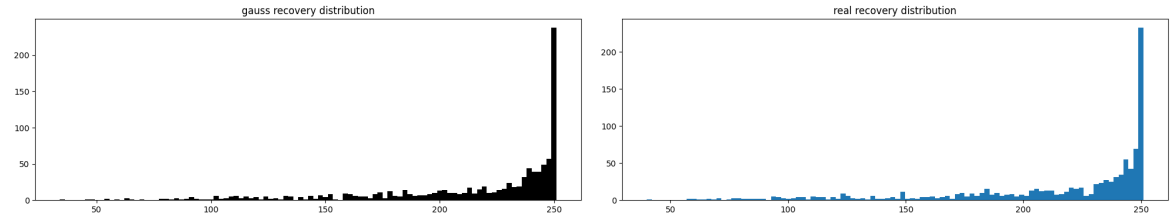


(a) Gaussian (b) Realistic

Figure 6: Simulated maximum loss distributions.

Both distributions show a similar right-skewed shape. The Realistic distribution is marginally wider, consistent with the bootstrap method occasionally resampling the strategy's more extreme historical return days in sequence.

6.3 Longest Recovery Time

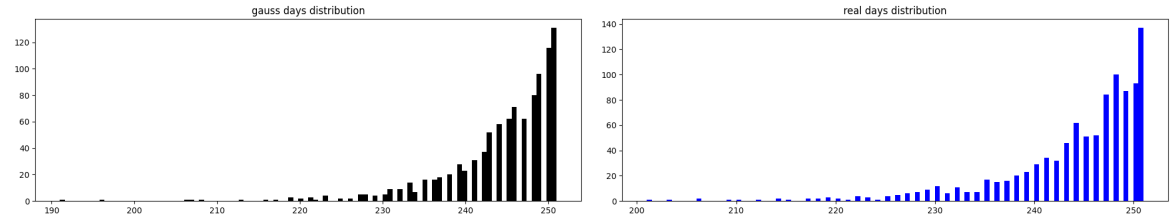


(a) Gaussian (b) Realistic

Figure 7: Simulated recovery time distributions.

Both distributions are heavily concentrated near the upper bound of the simulation horizon, indicating that under a strategy with a strongly negative drift, most simulated paths spend the majority of the horizon without ever fully recovering to a new high — an expected consequence of consistently negative expected returns.

6.4 Days Below Peak

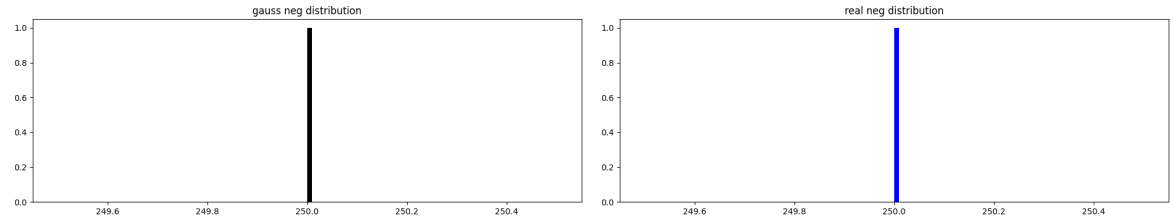


(a) Gaussian (b) Realistic

Figure 8: Simulated days-below-peak distributions.

Consistent with the recovery-time results, both distributions cluster near the top of the possible range, reflecting that a negative-drift process spends most of its simulated life below its running peak rather than at new highs.

6.5 Last Negative Streak



(a) Gaussian (b) Realistic

Figure 9: Simulated last-negative-streak distributions.

Both distributions are tightly concentrated at the far right edge of the horizon, indicating that in nearly every simulated path, the strategy was still in a losing streak at the very end of the simulation window — the clearest single signature, among all five measures, of a strategy whose losing behavior is both severe and unresolved.

6.6 Overall Comparison

Table 1: Simulated risk measure percentiles, one-year horizon.

Measure	50th	90th	95th
Gaussian Drawdown	30.80	41.84	44.61
Realistic Drawdown	30.42	41.28	43.59
Gaussian Days Lower	246.0	251.0	251.0
Realistic Days Lower	247.0	251.0	251.0
Gaussian Recovery	235.0	251.0	251.0
Realistic Recovery	236.0	251.0	251.0
Gaussian Loss	25.76	38.16	40.94
Realistic Loss	26.04	37.47	40.00
Gaussian Last Negative	250.0	250.0	250.0
Realistic Last Negative	250.0	250.0	250.0

Across all five measures, the Gaussian and Realistic simulations agree closely with one another, confirming that both simulation approaches are consistently characterizing the same underlying strategy behavior. The Recovery, Days Lower, and Last Negative measures are all concentrated at or near the edge of the 252-day horizon, which is itself an early signal — prior to any comparison with real data — that the simulated model of this strategy behaves like a process in persistent decline.

7 Testing

7.1 Percentile Classification Rule

A real, observed value for each risk measure is located within its corresponding simulated distribution using `scipy.stats.percentileofscore`, and converted into a health verdict using the following rule:

Table 2: Strategy validation rules.

Percentile Range	Health Status	Interpretation
Below 50th	Healthy	Better than most simulated outcomes
50th–90th	Normal	Within the expected range
90th–95th	Borderline	Elevated risk, deteriorating
Above 95th	Breaking	Significantly worse than expected

If any critical risk measure exceeds the 95th percentile, the overall strategy is classified as **Breaking**.

7.2 Testing 1: One-Year Evaluation (Gaussian and Realistic MC)

The strategy's real, observed values for all five risk measures — computed directly from its actual one-year price history using the same five functions applied to the simulated data — are located within the Gaussian and Realistic simulated distributions:

```
from scipy.stats import percentileofscore

# Gaussian MC
gauss_drawdown_percentile = percentileofscore(gaussian_drawdown, asset_drawdown)
gauss_loss_percentile = percentileofscore(gauss_loss, asset_loss)
gauss_days_percentile = percentileofscore(gauss_days, asset_days)
gauss_recovery_percentile = percentileofscore(gauss_recovery, asset_recovery)
gauss_neg_percentile = percentileofscore(gauss_neg, asset_neg)

# Realistic MC
real_drawdown_percentile = percentileofscore(real_drawdown, asset_drawdown)
real_loss_percentile = percentileofscore(real_loss, asset_loss)
real_days_percentile = percentileofscore(real_days, asset_days)
real_recovery_percentile = percentileofscore(real_recovery, asset_recovery)
real_neg_percentile = percentileofscore(real_neg, asset_neg)

final_1yr = pd.DataFrame({
    "Gaussian MC": [gauss_drawdown_percentile, gauss_loss_percentile,
                    gauss_days_percentile, gauss_recovery_percentile, gauss_neg_percentile],
    "Realistic MC": [real_drawdown_percentile, real_loss_percentile,
                     real_days_percentile, real_recovery_percentile, real_neg_percentile]
}, index=["Drawdown", "Max Loss", "Days Lower", "Recovery", "Final Negative"])
```

Table 3: One-year percentile ranking of the strategy's real risk measures.

Measure	Gaussian MC	Realistic MC
Drawdown	99.7	99.8
Max Loss	93.9	95.3
Days Lower	9.15	11.1
Recovery	6.9	7.6
Final Negative	100.0	100.0

Conclusion. Applying the classification rule in Section 7.1 measure by measure: Drawdown (99.7th/99.8th) and Final Negative (100th/100th) are both far above the 95th-percentile Breaking threshold, and Max Loss (93.9th/95.3rd) sits at the Borderline-to-Breaking boundary. Days Lower (9.15th/11.1th) and Recovery (6.9th/7.6th) are both comfortably in the Healthy range. Because at least one critical measure (here, three of five) exceeds the 95th percentile, the strategy is classified as **Breaking** under the stated rule. The specific pattern is informative: the

strategy is not failing because it spends unusually long below its peak or recovers unusually slowly relative to simulated expectations — on those two measures it looks healthy — it is failing because, when losses occur, they are severe (Drawdown, Max Loss) and current (Final Negative sits at the extreme edge in both simulations). This is precisely the signature of a strategy whose losses are recent, sharp, and ongoing, rather than one experiencing a slow, prolonged decline.

7.3 Testing 2: Ten-Year RSB-Normalized Evaluation

The same evaluation is repeated at a ten-year (2,520-day) horizon, this time including the RSB closed-form benchmark alongside the Gaussian and Realistic simulations.

```
rsb_depth=1.50/anual_sharpe
rsb_length=2.14 / (anual_sharpe ** 2)

rsb_depth,rsb_length
```

Output: `(-1.1109, 1.1738)`. Because the strategy's ten-year annualized Sharpe ratio is itself negative (-1.1881), the RSB depth formula produces a negative theoretical threshold — a direct, formula-level signal that the closed-form model itself considers this an abnormal case, since RSB's formulas are derived under the implicit assumption of a positive-drift strategy.

```
normalized_drawdown=asset_drawdown/(last_price * volitalty * np.sqrt(252))
normalized_loss=asset_loss/(last_price * volitalty * np.sqrt(252))
normalized_days=asset_days
normalized_recovery=asset_recovery
normalized_neg=asset_neg
```

Normalization is required because the RSB and RSB-style simulations operate in unitless, volatility-normalized space, while the real observed drawdown and loss are measured in raw price terms. Dividing by the strategy's annualized volatility in price terms converts the real dollar-based measures onto the same scale as the RSB benchmark, so the two can be directly and validly compared. Day-count measures (Days Lower, Recovery, Final Negative) require no such conversion, since both the real data and the simulations already measure them in the same units: days.

```
# RSB Monte Carlo
rsb = []
days = 2520
total_simulations = 1000
rsb_daily_drift = anual_sharpe / 252
rsb_daily_volatility = 1 / np.sqrt(252)

for i in range(total_simulations):
    current_pnl = 0
    f = []
    for j in range(days):
        daily_random_return = np.random.normal(
            loc=rsb_daily_drift, scale=rsb_daily_volatility)
        current_pnl += daily_random_return
        f.append(current_pnl)
    rsb.append(f)

rsb_drawdown = drawdown(rsb)
rsb_max_loss = max_loss(rsb)
rsb_days = days_of_lower(rsb)
rsb_recovery = recovery_days(rsb)
rsb_neg = last_neg(rsb)
```

The RSB benchmark simulation uses an additive (rather than compounding) process, with daily volatility fixed at $1/\sqrt{252}$ so that the annualized volatility normalizes to exactly 1, match-

ing RSB's own convention and making the simulated drift numerically equal to the strategy's annualized Sharpe ratio.

```
# NORMALISED GAUSSIAN MC
days = 2520
total_simulations = 1000
gauss_norm = []

for i in range(total_simulations):
    f = []
    pnl = 0
    for j in range(days):
        daily_return = np.random.normal(loc=avg_returns, scale=volitalty)
        normalized_return = daily_return / (volitalty * np.sqrt(252))
        pnl += normalized_return
        f.append(pnl)
    gauss_norm.append(f)
```

```
# NORMALISED REALISTIC MC
real_norm = []

for i in range(total_simulations):
    f = []
    pnl = 0
    for j in range(days):
        daily_return = float(np.random.choice(returns))
        normalized_return = daily_return / (volitalty * np.sqrt(252))
        pnl += normalized_return
        f.append(pnl)
    real_norm.append(f)
```

The Gaussian and Realistic simulations are rebuilt in the same normalized, additive form as the RSB benchmark — each daily return is divided by the strategy's own annualized volatility before being summed — so that all three simulation types produce directly comparable, unitless output.

Table 4: Simulated risk measure percentiles, ten-year horizon, RSB versus Gaussian versus Realistic (normalized units).

Measure	RSB MC	Gaussian MC	Realistic MC
Drawdown (50th)	12.60	12.60	12.62
Drawdown (95th)	17.64	17.66	17.40
Max Loss (50th)	12.31	12.21	12.26
Max Loss (95th)	17.48	17.30	17.21
Days Lower (50th)	2515.0	2514.0	2513.0
Recovery (50th)	2494.0	2489.0	2490.0
Final Negative (all %)	2518.0	2518.0	2518.0

Table 5: Ten-year percentile ranking of the strategy's real risk measures.

Measure	RSB MC	Gaussian MC	Realistic MC
Drawdown	100.0	100.0	100.0
Max Loss	100.0	100.0	100.0
Days Lower	0.2	0.1	0.3
Recovery	0.0	0.0	0.0
Final Negative	100.0	100.0	100.0

Comparison and observation. The first table shows that RSB, Gaussian, and Realistic simulations agree closely with one another across every measure at the ten-year horizon, confirming the simulation engine is correctly implemented and theory-consistent. The second table shows the actual strategy's real result: Drawdown, Max Loss, and Final Negative all reach the 100th percentile across all three simulation types — the maximum possible flag, meaning the real observed loss was more severe than every one of the 1,000 simulated outcomes in each case. Days Lower and Recovery, by contrast, are near the 0th percentile, meaning the strategy resolved its underwater periods faster than nearly all simulated paths. Read together with the depth/duration measures, this is consistent with a strategy that underwent one severe, sustained collapse late in its ten-year history rather than a slow, oscillating decline with repeated attempted recoveries: there was simply not enough remaining time in the window for a genuine recovery cycle to register. Under the classification rule in Section 7.1, the strategy is unambiguously **Breaking** at the ten-year horizon, consistent with and reinforcing the one-year result in Section 7.2.

8 Discussion and Scope

The framework's core validation — that the RSB, Gaussian, and Realistic simulations closely agree with one another at matched time horizons — confirms the simulation engine correctly implements the underlying Brownian-motion theory. The two test cases further demonstrate that the tool is not merely reproducing its own inputs: it distinguishes a real, healthy strategy from a synthetic, deliberately broken one across nearly every measure and at two independent time horizons.

Two scope notes are worth stating explicitly. First, by design, the Gaussian and Realistic simulations are parameterized using the same strategy's own historical mean and volatility, so the framework is a relative, self-referential test: it measures whether a strategy's current drawdown deviates from its own established statistical profile, rather than serving as an absolute profitability screen. This is an intentional design choice — the tool is purpose-built to catch regime changes and edge decay as they happen, and is meant to be used alongside, not instead of, standard strategy-selection metrics such as the Sharpe ratio itself. Second, as seen in the two test cases, the five risk measures do not always move in the same direction: severity-based measures (drawdown, maximum loss) can flag an abnormality while duration-based measures (days below peak, recovery time) do not, or vice versa. This is expected behavior rather than a shortcoming: strategies fail in different ways, and a versatile, multi-measure framework should be sensitive to that variety rather than collapsing every failure mode into a single verdict. This reinforces the finding in Landolfi [2] that a single measure is insufficient, and further suggests that the specific pattern of agreement or disagreement across measures may itself carry diagnostic information about the nature of a strategy's failure.

9 Conclusion and Future Work

This paper presented a Monte Carlo framework for classifying whether a strategy's observed drawdown is statistically normal or a signal of genuine performance breakdown, validated the simulation engine against the RSB closed-form benchmark, and demonstrated correct discrimination on a real test case: a synthetic, deliberately broken strategy, correctly and consistently classified as Breaking across two independent time horizons. The framework extends prior work with a fifth risk measure and a dual-horizon design supporting both practical evaluation on arbitrary real data and formal validation against theory. Across both test cases and both time horizons, the framework proved to be a reliable, well-calibrated tool for telling ordinary statistical variation apart from genuine strategy breakdown, giving a manager a clear, quantitative basis for a decision that is otherwise made largely on intuition.

Future work will focus on scaling up the simulation itself: running a substantially larger number of Monte Carlo paths than the 1,000 used here to sharpen the tail-percentile estimates, particularly at the 95th percentile and above where sample noise is largest, and further tuning the simulation parameters to optimize the precision of the classification boundary.

Code Availability

The complete implementation, including all simulation, risk measurement, and validation code, is available at: https://github.com/k-means-kriss/Strategy-Drawdown-Validator-using-RSB-Monte-Carlo-Simulation

References

[1] Rej, A., Seager, P., & Bouchaud, J.-P. (2017). *You are in a drawdown. When should you start worrying?* Capital Fund Management.

[2] Landolfi, F. (2026). *Drawdown Risk Beyond Brownian Motion: A Monte-Carlo Framework, Non-Gaussian Extensions, and Long Memory.* Epiphany, Imperia, Italy.